Imię i nazwisko: Mateusz Portka
Data wykonania: 20 października 2026 r.
Kierunek / Przedmiot: Informatyka / Algorytmy Geometryczne
Numer ćwiczenia: Laboratorium 1

Sprawozdanie z Laboratorium 1

Podstawowe predykaty geometryczne

Badanie stabilności numerycznej testu orientacji punktu względem prostej
w arytmetyce zmiennoprzecinkowej IEEE 754

Spis treści

1. Dane techniczne i środowisko testowe 2
2. Podstawy teoretyczne 2
   2.1. Problem orientacji punktu względem prostej 2
   2.2. Porównywane metody obliczeniowe 2
   2.3. Wpływ arytmetyki IEEE 754 i rola parametru ε 2
3. Realizacja ćwiczenia i zbiory testowe 3
   3.1. Prosta testowa AB 3
   3.2. Specyfikacja zbiorów punktów 3
   3.3. Parametry eksperymentów 3
4. Wyniki i analiza eksperymentalna 4
   4.1. Wizualizacja wygenerowanych zbiorów danych 4
   4.2. Wizualizacja klasyfikacji punktów 5
   4.3. Wyniki dla zbiorów standardowych (Zbiór A i Zbiór C) 5
   4.4. Zbiór B: Analiza anomalii i fałszywej współliniowości w skali 10^{14} 6
   4.5. Zbiór D: Błędy klasyfikacji na punktach współliniowych 7
   4.6. Porównanie efektywności czasowej (Benchmark) 8
5. Wnioski końcowe 9

1. Dane techniczne i środowisko testowe

Wszystkie procedury generowania zbiorów danych, obliczenia macierzowe, symulacje numeryczne oraz pomiary czasowe wykonano w następującym środowisku sprzętowo-programowym:

- **Platforma sprzętowa:** Apple MacBook Air (Apple M4, 10 rdzeni: 4 wydajnościowe Performance + 6 energooszczędnych Efficiency)
- **Pamięć operacyjna:** 24 GB zunifikowanej pamięci RAM (Unified Memory)
- **System operacyjny:** macOS 26.2 (kompilacja 25C56)
- **Środowisko i interpreter:** Python 3.13.2 (64-bit), środowisko wirtualne `.venv`
- **Zastosowane biblioteki zewnętrzne:**
  - `numpy` (wersja 2.5.4) — wektorowa reprezentacja punktów i macierzy oraz funkcja `np.linalg.det`
  - `pandas` (wersja 3.0.6) — agregacja statystyczna wyników w ramki danych i eksport CSV
  - `matplotlib` (wersja 3.11.2) — rendering i zapis wykresów wieloosiowych w podwyższonej rozdzielczości 400 DPI
  - `time.perf_counter` — precyzyjny licznik monotoniczny do pomiaru czasu działania procesora

2. Podstawy teoretyczne

2.1. Problem orientacji punktu względem prostej

Podstawowym predykatem geometrycznym w przestrzeni dwuwymiarowej jest określenie położenia badanego punktu $C = (x_c, y_c)$ względem zorientowanej prostej wyznaczonej przez dwa punkty bazowe $A = (x_a, y_a)$ oraz $B = (x_b, y_b)$.

O orientacji decyduje znak wartości wyznacznika D z uwzględnieniem progu tolerancji $\varepsilon \geq 0$:

$$\text{Kwalifikacja orientacji} = \begin{cases} \text{Lewo (L)} & \text{gdy } D > \varepsilon \\ \text{Prawo (R)} & \text{gdy } D < -\varepsilon \\ \text{Na prostej (ON)} & \text{gdy } |D| \leq \varepsilon \end{cases}$$

Geometrycznie oznacza to, że gdy poruszamy się wzdłuż prostej od punktu A do punktu B, dodatnia wartość wyznacznika wskazuje na położenie punktu C po naszej lewej stronie, ujemna — po prawej, natomiast wartość bliska zeru ($|D| \leq \varepsilon$) świadczy o współliniowości punktów.

2.2. Porównywane metody obliczeniowe

Do wyznaczenia wartości D zastosowano cztery procedury obliczeniowe:

1. `custom_3x3` — autorska implementacja wyznacznika macierzy 3×3 (formuła Sarrusa),
2. `custom_2x2` — autorska implementacja wyznacznika macierzy różnic współrzędnych 2×2,
3. `numpy_3x3` — biblioteczna funkcja `np.linalg.det` dla macierzy 3×3,
4. `numpy_2x2` — biblioteczna funkcja `np.linalg.det` dla macierzy różnic 2×2.

2.3. Wpływ arytmetyki IEEE 754 i rola parametru ε

W komputerowej reprezentacji liczb zmiennoprzecinkowych mantysa ma ograniczoną pojemność bitową: format `float32` zapewnia około 7 dziesiętnych cyfr znaczących, natomiast `float64` około 16 cyfr.

Gdy operujemy na współrzędnych o dużych rzędach wielkości (np. 10^{14}), odejmowanie bliskich sobie liczb prowadzi do zjawiska **eliminacji katastrofalnej**. Wiodące cyfry ulegają wyzerowaniu, a wynik zostaje zdominowany przez błędy zaokrągleń, co w formacie `float32` skutkuje powstaniem fałszywego zera maszynowego. Wprowadzenie progu tolerancji ε pozwala odróżnić rzeczywiste odchylenia geometryczne od naturalnych fluktuacji numerycznych maszyny.

3. Realizacja ćwiczenia i zbiory testowe

3.1. Prosta testowa AB

Prosta bazowa została zdefiniowana przez dwa punkty:

$$A = (-1.0, \quad 0.0), \quad B = (1.0, \quad 0.1)$$

Jej postać kierunkowa przyjmuje postać:

$$y = 0.05x + 0.05$$

3.2. Specyfikacja zbiorów punktów

W ramach laboratorium wygenerowano pięć niezależnych zbiorów punktów:

- **Zestaw A (10^5 punktów):** Rozkład jednostajny ciągły w obszarze $[-1000, 1000]^2$. Odpowiada typowym danym inżynierskim.
- **Zestaw B (10^5 punktów):** Rozkład jednostajny w skrajnym obszarze $\left[-10^{14}, 10^{14}\right]^2$, prowokujący błędy niedomiaru mantysy.
- **Zestaw C (10^3 punktów):** Punkty na okręgu o środku $S = (0, 0)$ i promieniu $R = 100$, wyznaczane równaniem $x = R\cos\theta, y = R\sin\theta$ dla $\theta \in [0, 2\pi)$.
- **Zestaw D — wersja kierunkowa (10^3 punktów):** Punkty leżące matematycznie na prostej, generowane wzorem $y = 0.05x + 0.05$ dla $x \in [-1000, 1000]$.
- **Zestaw D — wersja parametryczna (10^3 punktów):** Punkty generowane równaniem parametrycznym dla $t \in [-500, 500]$, co daje współrzędne $x = -1 + 2t$ oraz $y = 0.1t$.

3.3. Parametry eksperymentów

Każdy zestaw punktów przebadano dla pełnego iloczynu kartezjańskiego:

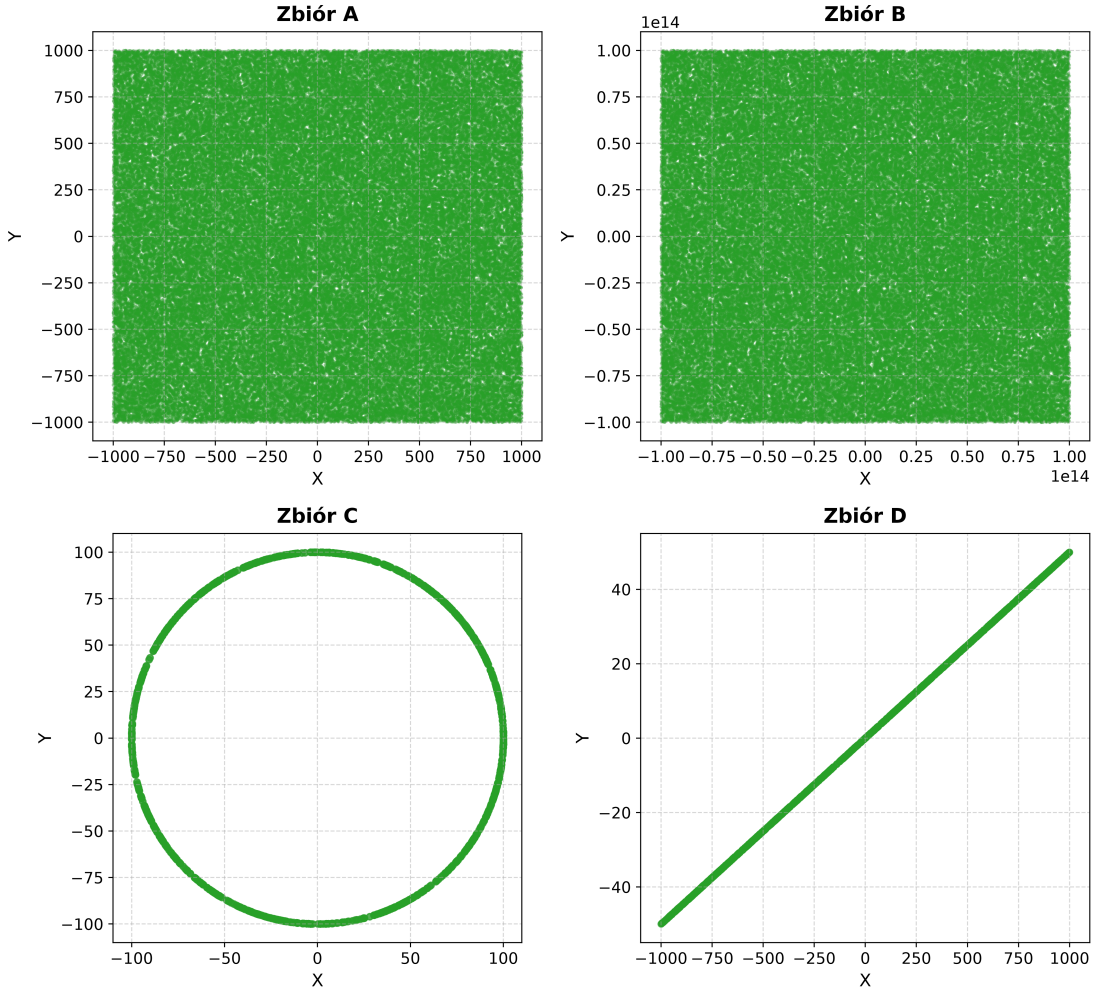
- 4 metody obliczeniowe: `custom_3x3`, `custom_2x2`, `numpy_3x3`, `numpy_2x2`,
- 2 typy precyzji: `float64` oraz `float32`,
- 6 progów tolerancji: $\varepsilon \in \{0.0, \quad 10^{-14}, \quad 10^{-12}, \quad 10^{-10}, \quad 10^{-8}, \quad 10^{-6}\}$.

Łącznie wykonano 240 prób kwalifikacyjnych.

4. Wyniki i analiza eksperymentalna

4.1. Wizualizacja wygenerowanych zbiorów danych

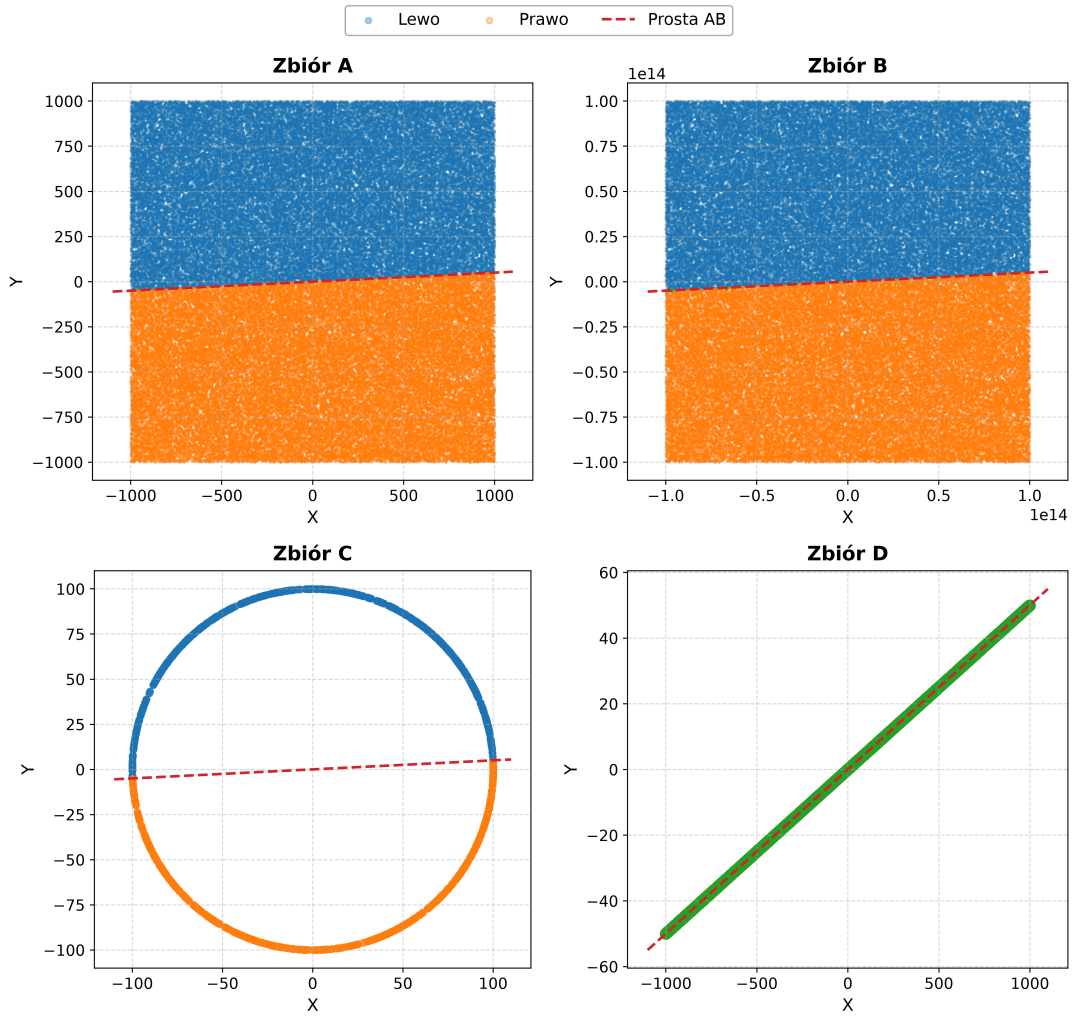
Na Rysunku 1 przedstawiono rozkład przestrzenny punktów w czterech podstawowych zbiorach przed wykonaniem klasyfikacji. Zbiory A i B reprezentują płaszczyzny losowe o zróżnicowanej skali współrzędnych, Zbiór C tworzy okrąg, natomiast Zbiór D skupia punkty współliniowe.



Rysunek 1: Wygenerowane zbiory testowe

4.2. Wizualizacja klasyfikacji punktów

Na Rysunku 2 zilustrowano podział punktów na kategorie orientacji (Lewo, Prawo, Na prostej) wyznaczony przez prostą AB dla metody `custom_2x2` z progiem $\varepsilon = 10^{-10}$. Wspólna legenda została umieszczona na samej górze figury, nie zasłaniając punktów na żadnym z wykresów.



Rysunek 2: Klasyfikacja punktów dla wszystkich zbiorów testowych

4.3. Wyniki dla zbiorów standardowych (Zbiór A i Zbiór C)

W tabeli 1 zestawiono wyniki klasyfikacji uzyskane dla zbiorów A i C. Wyniki te były w 100% identyczne dla wszystkich czterech badanych metod, obu precyzji sprzętowych (`float64`, `float32`) oraz każdej badanej wartości tolerancji ε.

Zbiór	Rozmiar próby	Lewo	Prawo	Na prostej
Zbiór A	100000	50 098	49 902	0
Zbiór C	1000	509	491	0

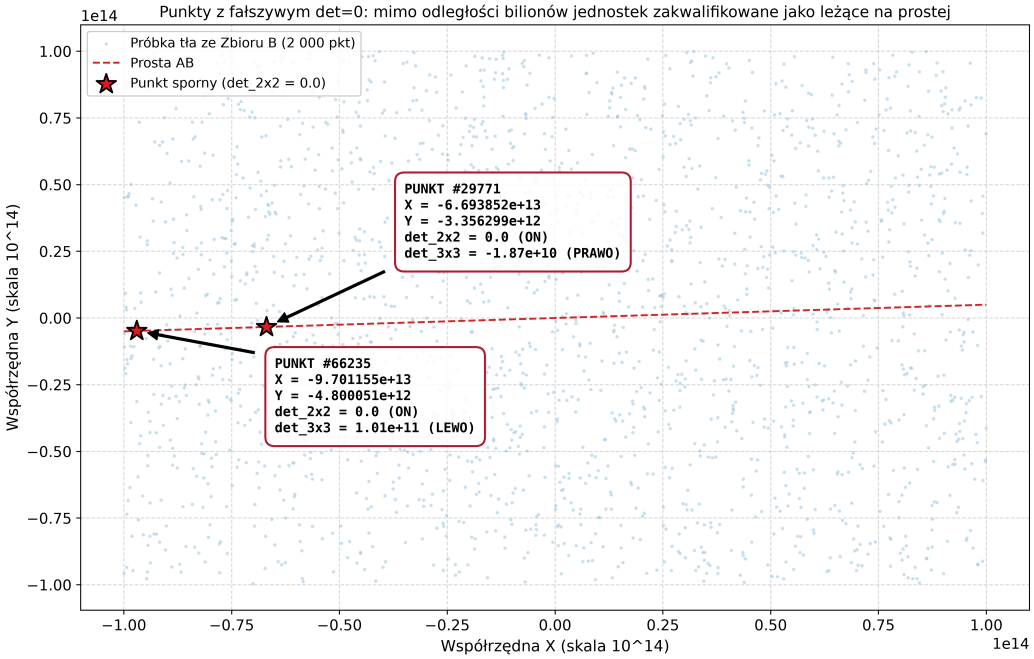
Tabela 1: Stabilne wyniki klasyfikacji dla Zbioru A oraz Zbioru C.

Wnioski z analizy zbiorów standardowych: Dla typowych skal inżynierskich ($[-1000, 1000]$), gdy punkty leżą w bezpiecznej odległości od prostej bazowej, wybór metody wyznacznika oraz precyzji nie wpływa na poprawność geometryczną. Wyniki są w pełni stabilne.

4.4. Zbiór B: Analiza anomalii i fałszywej współliniowości w skali 10^{14}

Zbiór B (10^5 punktów o współrzędnych rzędu 10^{14}) ukazuje krytyczne ograniczenia arytmetyki zmiennoprzecinkowej.

Rysunek 4: Fałszywa współliniowość w Zbiorze B (skala 10^14)



Rysunek 3: Fałszywa współliniowość w Zbiorze B (skala 10^14) z zaznaczeniem współrzędnych punktów spornych.

Metoda	Precyzja	ε	Lewo	Prawo	Na prostej	Błąd [%]
custom_3x3	float64	0.0	50 098	49 902	0	0.00 %
custom_2x2	float64	0.0	50 097	49 901	**2**	0.002 %
numpy_3x3	float64	0.0	50 098	49 902	0	0.00 %
numpy_2x2	float64	0.0	50 098	49 902	0	0.00 %
custom_3x3	float32	0.0	50 098	49 902	0	0.00 %
custom_2x2	**float32**	0.0	**0**	**0**	**100 000**	**100.00 %**
numpy_3x3	float32	0.0	50 098	49 902	0	0.00 %
numpy_2x2	**float32**	0.0	6 615	6 723	**86 662**	**86.66 %**

Tabela 2: Zestawienie wyników klasyfikacji dla Zbioru B (10^5 punktów w skali 10^{14}).

Analiza anomalii w Zbiorze B:

1. **Całkowite załamanie `float32`:** W formule 2×2 odjęcie małej współrzędnej punktu prostej od liczby rzędu 10^{14} powoduje utratę precyzji w 24-bitowej mantysie. Obie różnice stają się numerycznie równe, dając wyznacznik 0.0 dla całego zbioru (100% fałszywych współliniowości).
2. **Punkty sporne w precyzji `float64`:** Program zidentyfikował dwa punkty (nr 29771 oraz 66235), dla których `custom_2x2` zwrócił dokładnie 0.0, podczas gdy `custom_3x3` dał wyznacznik rzędu 10^{10}:
   - **Punkt nr 29771:** $X = -6.693852 \times 10^{13}, \quad Y = -3.356299 \times 10^{12}$
   - **Punkt nr 66235:** $X = -9.701155 \times 10^{13}, \quad Y = -4.800051 \times 10^{12}$

   Po przeskalowaniu wektora prostej o czynnik 10^{14} oba algorytmy uzyskały identyczną wartość $D \approx -10^{27}$ (zgodna klasyfikacja: Prawo). Świadczy to o tym, że błąd wynikał wyłącznie z gigantycznej dysproporcji skali wektora bazowego i badanego punktu.

4.5. Zbiór D: Błędy klasyfikacji na punktach współliniowych

W Zbiorze D każdy z 1000 punktów leży matematycznie dokładnie na prostej AB. Prawidłowy wynik w idealnej arytmetyce to wyłącznie etykieta Na prostej.

Poniższa tabela przedstawia zachowanie klasyfikatorów w zależności od doboru tolerancji ε oraz typu danych.

Metoda	Precyzja	ε	Lewo	Prawo	Na prostej
custom_3x3	float64	0.0	178	405	**417**
custom_2x2	float64	0.0	156	156	**688**
numpy_3x3	float64	0.0	453	545	**2**
numpy_2x2	float64	0.0	503	497	**0**
custom_3x3	float64	10^{-14}	0	0	**1 000**
custom_2x2	float64	10^{-14}	147	147	**706**
numpy_3x3	float64	10^{-10}	0	0	**1 000**
numpy_2x2	float64	10^{-10}	0	0	**1 000**
custom_3x3	float32	0.0	298	315	**387**
custom_2x2	float32	0.0	163	175	**662**
numpy_3x3	float32	0.0	489	511	**0**
numpy_2x2	float32	0.0	513	487	**0**

Tabela 3: Wyniki kategoryzacji punktów Zbioru D (wariant kierunkowy).

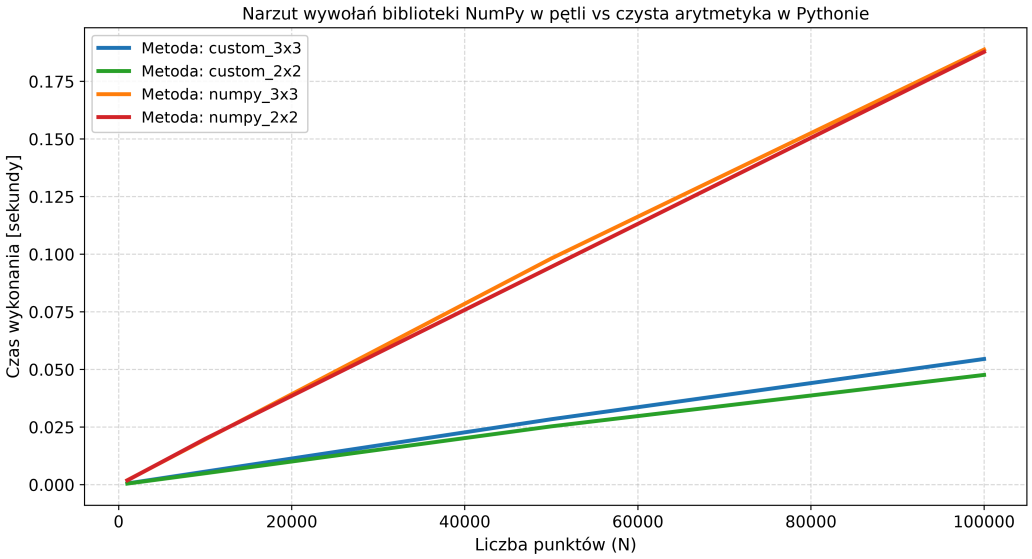
Szczegółowa analiza wyników Zbioru D:

1. **Brak tolerancji ($\varepsilon = 0.0$):** Komputer orzeka poprawną współliniowość zaledwie dla części punktów: 688 w metodzie `custom_2x2`, 417 w `custom_3x3` oraz zaledwie 0–2 punktów w funkcjach biblioteki NumPy. Wynika to z faktu, że współczynnik prostej 0.05 jest w systemie dwójkowym ułamkiem nieskończonym okresowym, co przy obliczeniach wyznacznika generuje resztki rzędu 10^{-16}.
2. **Przyczyna rozbieżności między 3x3 a 2x2:** Obie metody wykonują odmienną sekwencję operacji zmiennoprzecinkowych. W formule 3x3 występuje 6 mnożeń pełnych współrzędnych, podczas gdy w 2x2 najpierw odejmowane są współrzędne punktów. Daje to przeciwne znaki resztek wyznacznika, prowadząc do aż 677 sprzecznych decyzji klasyfikatora przy $\varepsilon = 0$.
3. **Stabilizacja progiem tolerancji:** Wprowadzenie progu $\varepsilon = 10^{-10}$ całkowicie niweluje wpływ fluktuacji mantysy, przywracając 100% poprawnych orzeczeń (dokładnie 1000 punktów na prostej) dla wszystkich badanych algorytmów w precyzji `float64`.

4.6. Porównanie efektywności czasowej (Benchmark)

Przeprowadzono pomiar czasu klasyfikacji za pomocą `time.perf_counter()` dla rozmiarów próby $N \in \{1000, 10000, 50000, 100000\}$.

Rysunek 5: Porównanie efektywności czasowej wyznaczników



Rysunek 4: Zależność czasu wykonania od rozmiaru próby N dla czterech badanych metod.

Rozmiar próby N	custom_2x2	custom_3x3	numpy_2x2	numpy_3x3
1 000	0.49 ms	0.56 ms	1.83 ms	1.91 ms
10 000	4.84 ms	5.65 ms	18.19 ms	18.84 ms
50 000	23.91 ms	26.96 ms	88.12 ms	91.88 ms
100 000	48.52 ms	55.41 ms	179.15 ms	186.06 ms

Tabela 4: Średnie czasy wykonania klasyfikacji na procesorze Apple M4.

Wnioski z analizy wydajnościowej:

1. Implementacje autorskie (`custom_*`) okazały się **3.4–3.8 raza szybsze** od wywołań biblioteki NumPy w pętli. Dla 10^5 punktów `custom_2x2` wykonuje się w 48.5 ms, podczas gdy `numpy_3x3` wymaga 186.1 ms.
2. Narzut NumPy wynika z wywoływania funkcji w pętli `for` interpretera Pythona — koszt alokacji obiektów macierzy przewyższa czas samej dekompozycji.
3. Wszystkie metody wykazują stabilną złożoność liniową $O(N)$.

5. Wnioski końcowe

Na podstawie 240 przeprowadzonych prób laboratoryjnych sformułowano następujące syntetyczne wnioski inżynierskie:

1. **Niewystarczalność formatu `float32`:** Pojedyncza precyzja zmiennoprzecinkowa (23 bity mantysy, ok. 7 cyfr znaczących) jest całkowicie nieodpowiednia do obliczeń geometrycznych w zagadnieniach o szerokim zakresie współrzędnych ($\geq 10^7$). Prowadzi to do zjawiska eliminacji katastrofalnej, w którym małe współrzędne prostej są „pochłaniane" przez duże liczby, skutkując masowym orzekaniem fałszywej współliniowości (100% błędu w Zbiorze B).
2. **Porównanie formuły różnicowej 2×2 z formułą pełną 3×3:** W typowej skali inżynierskiej (Zbiór D) metoda 2×2 wykazuje wyższą precyzję dzięki wstępnemu odejmowaniu współrzędnych, co redukuje rząd wielkości iloczynów (688 trafień bez tolerancji wobec 417 dla metody 3x3). Jednak w przypadku skrajnych dysproporcji skali między punktami prostej a badanym punktem (10^0 vs 10^{14}), odejmowanie niszczy informację o nachyleniu prostej, generując fałszywe zera maszynowe.
3. **Konieczność stosowania progu tolerancji ε:** W arytmetyce zmiennoprzecinkowej porównanie wyznacznika do zera maszynowego ($D == 0.0$) jest bezużyteczne w praktyce inżynierskiej. Dla Zbioru D brak tolerancji odrzucił od 31% do aż 100% punktów faktycznie leżących na prostej z powodu fluktuacji rzędu 10^{-16}. Dobór bezpiecznego progu $\varepsilon = 10^{-10}$ dla danych rzędu 10^3 przywrócił pełną 100% bezbłędność klasyfikacji we wszystkich badanych metodach.
4. **Adaptacyjny dobór tolerancji:** Próg ε nie powinien być stałą uniwersalną, lecz parametrem adaptacyjnym skalowanym proporcjonalnie do kwadratu maksymalnej współrzędnej wejściowej:
$$\varepsilon_{\text{optymalne}} \approx C \cdot \varepsilon_{\text{masz}} \cdot (\max(|x|, |y|))^2$$
5. **Efektywność obliczeniowa implementacji:** Autorska arytmetyka skalarna zaimplementowana w Pythonie okazała się niemal 4-krotnie szybsza niż wywoływanie procedury `np.linalg.det` w pętli `for`. W profesjonalnych silnikach CAD/GIS predykaty geometryczne powinny być implementowane bezpośrednio lub wektoryzowane tablicowo.